

T03 — Compilazione, Esecuzione e Deployment

JVM, Bytecode, JDK vs JRE, Strumenti CLI (javac, java, jar) e Apache Maven

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo l’ecosistema tecnologico di Java, l’architettura della **Java Virtual Machine (JVM)**, i meccanismi di compilazione, il **Dynamic Class Loading**, la gestione del **Classpath** e le strategie di impacchettamento con file **JAR** ed eseguibili ([T03 - Building, Running and Deployment in Java.pdf](#)).

0.0.1 1. Teoria: Concetti Teorici dalle Slide

A. Il Linguaggio Java e le sue Proprietà Fondamentali (Slide 1–11)

- **Cenni Storici:**

- Creato da **James Gosling** presso Sun Microsystems (progetto *Oak* nel 1991, rilasciato ufficialmente come Java nel **1995**, poi acquisito da Oracle).
- Motto: “*Write Once, Run Everywhere*” (WORA).

- **Le Caratteristiche Chiave del Linguaggio:**

1. **Robustezza:**

- *Tipizzazione forte e statica (Strongly Typed)*: ogni variabile ed espressione ha un tipo noto a tempo di compilazione; i vincoli di tipo sono verificati dal compilatore prima dell’esecuzione.
- *Nessuna manipolazione esplicita di puntatori*: in Java non esistono l’operatore di indirizzo `&`, l’aritmetica dei puntatori né la deallocazione manuale (`free()`). Questo elimina alla radice *dangling pointers*, *segmentation fault* e *buffer overflow*.
- *Controlli a runtime*: la JVM esegue verifiche automatiche su ogni accesso ad array (`ArrayIndexOutOfBoundsException`) e sui riferimenti (`NullPointerException`).
- *Garbage Collection (GC) automatica*: un thread demone a bassa priorità della JVM individua ed elimina gli oggetti non più raggiungibili nello Heap, riducendo drasticamente i *memory leak*.
- *Gestione strutturata degli errori*: gestione degli stati anomali tramite eccezioni controllate e non controllate (`try-catch-finally`).

2. **Dinamicità:**

- *Caricamento e linking dinamico (Dynamic Linking)*: le classi non vengono collegate in un unico binario monolitico a compile-time, ma sono caricate in memoria dalla JVM *on-demand*, solo quando effettivamente referenziate dal codice in esecuzione.
- *Allocazione dinamica*: la dimensione delle strutture dati e degli array può essere determinata a runtime (`new int[size]`).

3. Portabilità e Architettura a Bytecode:

- Nei linguaggi puramente compilati (C/C++), il compilatore genera direttamente codice macchina specifico per l'architettura target (x86, x64, ARM) e per il sistema operativo (Linux, Windows, macOS). Il programmatore deve ricompilare o mantenere build cross-platform separate.
- In Java, il compilatore (javac) traduce il sorgente .java in un formato intermedio indipendente dall'hardware: il **Bytecode** (.class).
- È la **JVM** a farsi carico della traduzione del bytecode in istruzioni macchina della CPU ospitante. La portabilità cessa di essere un onere del programmatore e diventa un servizio fornito dal runtime.

4. Prestazioni e Compilatore JIT (Just-In-Time):

- Java non è puramente interpretato. La JVM include un compilatore **JIT** che monitora a runtime le porzioni di bytecode eseguite più frequentemente (*hot spots*) e le compila al volo in codice macchina nativo ottimizzato, memorizzandole nella cache del codice.

• Java vs JavaScript:

- Citazione di Christian Heilmann: «*Java is to JavaScript what Car is to Carpet*» (hanno in comune solo le prime quattro lettere). JavaScript nacque da Netscape con quel nome per motivi puramente commerciali; i due linguaggi non condividono né modello dei tipi, né macchina virtuale, né filosofia di design.

B. Ecosistema Java: JDK vs JRE (Slide 11) La distinzione tra ambiente di sviluppo e ambiente di esecuzione:

- **JDK (Java Development Kit)**: ambiente per programmatori:
- javac: il compilatore che traduce sorgenti .java in bytecode .class.
- java: il launcher applicativo che avvia la JVM.
- javadoc: generatore di documentazione HTML a partire dai commenti speciali `/** ... */`.
- javap: disassemblatore di bytecode (permette di ispezionare il bytecode generato).
- jdb: debugger da riga di comando.
- **JRE (Java Runtime Environment)**: ambiente minimale per l'utente finale:
- **JVM (Java Virtual Machine)**: il motore di esecuzione.
- Librerie standard delle API Java (il modulo base java.base contenente java.lang, java.util, java.io, ecc.).
- Compilatore JIT integrato.

C. Compilazione, Esecuzione e Dynamic Class Loading (Slide 12–16)

1. La Struttura del Programma Minimo:

```

1 // Salvato obbligatoriamente in MyClass.java (stesso nome della classe pubblica)
2 public class MyClass {
3     public static void main(String[] args) {
4         System.out.println("Hello World!");
5     }
6 }

```

- **Regola aurea:** In un file .java può essere presente al massimo **una sola classe pubblica**, e il nome del file deve coincidere esattamente (incluso il maiuscolo/minuscolo) con il nome di tale classe.
- **Firma del metodo main:**
 - public: deve essere accessibile dall'esterno da parte del launcher della JVM.
 - static: la JVM lo invoca direttamente sulla classe, senza dover prima istanziare un oggetto (MyClass obj = new MyClass()).
 - void: non restituisce un codice numerico di ritorno (l'uscita anomala si gestisce con System.exit(code)).

- `String[] args`: array di stringhe contenente i parametri passati da riga di comando.

2. Pipeline di Esecuzione della JVM:

Sorgente (`.java`) $\xrightarrow{\text{javac}}$ Bytecode (`.class`) $\xrightarrow{\text{Loader}}$ Bytecode Verifier $\xrightarrow{\text{Interpreter / JIT}}$ OS / Hardware

- **Bytecode Verifier**: componente di sicurezza fondamentale della JVM. Prima di eseguire qualsiasi file `.class`, scansiona le istruzioni per verificare che non violino i limiti dello stack, non convertano puntatori illegalmente e rispettino le regole di visibilità.

3. Dynamic Class Loading e Classpath (`-cp`):

- La JVM non cerca i file nel filesystem a caso: utilizza il **Classpath** (una lista ordinata di directory e archivi `.jar`).
- Quando durante l'esecuzione il codice fa riferimento per la prima volta a una classe `x`:
 1. Il `ClassLoader` cerca `x.class` nella prima cartella specificata nel Classpath.
 2. Se non la trova, passa alla successiva.
 3. Se la trova, la carica in memoria e ne inizializza le strutture statiche.
 4. Se non la trova in nessuna cartella del Classpath, solleva `ClassNotFoundException` o `NoClassDefFoundError`.
- Esempio di esecuzione esplicita:

```
1 java -cp . MyClass
```

Il parametro `-cp .` istruisce la JVM a cercare le classi compilate nella directory corrente (`.`). Le classi standard di sistema (`System`, `String`) vengono invece caricate automaticamente dal bootstrap classloader della JDK.

D. Deployment e Gestione dei File JAR (Slide 17–18)

• Anatomia di un file JAR (*Java Archive*):

- Un file `.jar` è fisicamente un **archivio compresso in formato standard ZIP**.
- Raggruppa decine o centinaia di file `.class`, risorse (immagini, file di configurazione) e metadati.

• Creazione manuale da CLI:

```
1 jar cvf MyJar.jar MyClass.class
```

- `c`: *create* (crea nuovo archivio).
- `v`: *verbose* (mostra i file aggiunti).
- `f`: *file* (il parametro successivo è il nome del file `.jar` da produrre).

• Esecuzione tramite Classpath:

```
1 java -cp MyJar.jar MyClass
```

• JAR Auto-Eseguibile (`java -jar`):

- Per consentire all'utente di lanciare l'archivio direttamente con `java -jar MyJar.jar`, il JAR deve specificare quale classe ospita il metodo `main`.
- Questa informazione risiede nel file `META-INF/MANIFEST.MF` sotto la direttiva:

```
1 Main-Class: MyClass
```

- Creazione con file manifest esterno:

```
1 jar cvfm MyClass.jar manifest.txt MyClass.class
```

- Creazione con sintassi moderna senza file di testo intermedio (flag e per entypoint):

```
1 jar cfe MyClass.jar MyClass MyClass.class
```

E. Strumenti di Sviluppo: Da Editor a IntelliJ IDEA (Slide 19–26)

- **CLI vs IDE:** per progetti complessi, l'uso del terminale e di un editor di testo semplice diventa poco scalabile.
- **Vantaggi dell'IDE (*Integrated Development Environment*):**
 - *Syntax highlighting* e *live code inspection/linting* (segnalazione immediata degli errori di tipo e code smell prima ancora di compilare).
 - *Debugger integrato* con breakpoint, ispezione dello stack dei frame e monitoraggio variabili.
 - *Integrazione con Version Control (Git)* e *Build Automation Tools (Maven / Gradle)*.
 - *Monito del docente:* l'auto-completamento o la generazione automatica di codice da parte dell'IDE è utile in ambito professionale, ma va limitata durante la fase di apprendimento per comprendere a fondo le regole sintattiche e semantiche del linguaggio.
- **Scelta del corso: IntelliJ IDEA** (gratuito in versione Community o con licenza Education per studenti universitari).

0.0.2 2. Codice: Collegamento Pratico con il Tuo Progetto: Maven e guida-jar.md

Nel tuo workspace [esercizi-wally-25-26](#), hai formalizzato questi esatti concetti in modo eccellente in [guida-jar.md](#) e nei file di configurazione Maven:

1. Il Fully Qualified Name e il Filesystem:

- Come visto a lezione, se una classe appartiene al package `es01`, il bytecode generato deve risiedere fisicamente nel percorso `es01/Date.class`.
- Quando si invoca la JVM, il nome della classe da passare deve essere il FQN (`es01.MainDate`).

2. Automazione del Packaging con Maven:

- Invece di lanciare manualmente `jar cfe`, nei singoli moduli (es. [es01/pom.xml](#)) usi il plugin `maven-assembly-plugin`:

```
1 <plugin>
2   <groupId>org.apache.maven.plugins</groupId>
3   <artifactId>maven-assembly-plugin</artifactId>
4   <version>3.7.1</version>
5   <configuration>
6     <archive>
7       <manifest>
8         <mainClass>es01.MainDate</mainClass>
9       </manifest>
10    </archive>
11    <descriptorRefs>
12      <descriptorRef>jar-with-dependencies</descriptorRef>
13    </descriptorRefs>
14  </configuration>
15  <executions>
```

```
16     <execution>
17         <id>make-assembly</id>
18         <phase>package</phase>
19         <goals>
20             <goal>single</goal>
21         </goals>
22     </execution>
23 </executions>
24 </plugin>
```

Questo genera automaticamente nella cartella `target/` il cosiddetto *Fat JAR* contenente sia il bytecode dell'applicazione sia il file `META-INF/MANIFEST.MF` precompilato.

0.0.3 3. Test: Verifica dei Comandi da Terminale

Puoi verificare l'intero ciclo di build ed esecuzione da riga di comando testando il modulo `es01` tramite lo script:

```
1 ./esercizi.sh package es01
```

Lo script esegue internamente: 1. `mvn clean package -pl es01` 2. Individua il file JAR autoprodotta in `es01/target/es01-*-jar-with-dependencies.jar` 3. Esegue `java -jar ...` verificando che l'entrypoint `es01.MainDate` risponda correttamente.

Nota di Studio

Con T03 abbiamo concluso l'intera panoramica propedeutica (Introduzione, Paradigmi/UML e Tooling/JVM).

A partire dal prossimo blocco entriamo nel codice sorgente vero e proprio con la **Lezione 4 / Slide T04: *Types and Objects*** e il file ufficiale del docente [Example.java](#): - Tipi primitivi (dimensioni, range, valori di default) vs Reference Types. - Rappresentazione dei caratteri (`char` come intero senza segno a 16 bit, codifica ASCII e Unicode `\u0056`). - Regole di inizializzazione: variabili di istanza/statiche (default zero/null) vs variabili locali dello stack (nessun default → errore a compile-time!). - Operatori logici standard (`&`, `|`) vs operatori a corto circuito (*short-circuit* `&&`, `||`) e relativi effetti collaterali (`k++`). - Scope dei blocchi di codice annidati (`{ ... }`) e shadowing/visibilità delle variabili.

Dammi conferma per aprire la Lezione 4 / T04 e analizzare teoria e codice di `Example.java`!